

Administration Linux

Programmation Bash

Victor OYETOLA, Eric ATTOU

Université d'Abomey-Calavi

October 28, 2024

- 1 Introduction
- 2 Les variables et arguments
 - Les variables
 - Les arguments
- 3 La sous-exécution
- 4 L'arithmétique
- 5 Les structures de contrôle
 - if/then/else/fi
 - La commande test
 - Le branchement case et select
- 6 Les boucles
 - La boucle for
 - La boucle until
 - La boucle while
- 7 Les fonctions

Commande	Rôle
cat	Affiche le contenu d'un fichier
grep	Filtrage par expressions régulières basiques
egrep	Filtrage par expressions régulières étendues
tr	Transpose ou supprime de caractères dans une chaîne
pr	Affiche un fichier avec n colonnes et m lignes par page
paste	Concatène plusieurs fichiers ou lignes en un seul
head	Affiche les premières lignes d'un fichier
tail	Affiche les dernières lignes d'un fichier
cut	Sélectionne certains caractères dans une chaîne

Table: quelques commandes Linux et leurs rôles (1/2)

sed	Supprime ou remplace une partie d'une chaîne
awk	Langage script pour le traitement avancé de fichiers texte
sort	Trie les lignes d'un fichier
uniq	Supprime les doublons dans un fichier trié
nl	Affiche un fichier en numérotant les lignes
split	Découpe un fichier en plusieurs fichiers
date	Affiche une date formatée
for	Répète une commande plusieurs fois
seq	Génère une liste de nombres

Table: quelques commandes Linux et leurs rôles (2/2)

Commande	Description
<code>cat fichier.txt</code>	Affiche le contenu de "fichier.txt".
<code>grep "motif" fichier.txt</code>	Recherche "motif" dans "fichier.txt".
<code>egrep "(motif1 motif2)" fichier.txt</code>	Recherche "motif1" ou "motif2" dans "fichier.txt" avec des expressions régulières étendues.
<code>echo "hello world" tr 'a-z' 'A-Z'</code>	Convertit les minuscules en majuscules dans la chaîne "hello world".
<code>pr -3 fichier.txt</code>	Affiche "fichier.txt" en 3 colonnes.
<code>paste fichier1.txt fichier2.txt</code>	Concatène "fichier1.txt" et "fichier2.txt" côte à côte.
<code>head -n 5 fichier.txt</code>	Affiche les 5 premières lignes de "fichier.txt".
<code>tail -n 5 fichier.txt</code>	Affiche les 5 dernières lignes de "fichier.txt".
<code>cut -d',' -f1 fichier.csv</code>	Sélectionne le premier champ (séparé par des virgules) dans "fichier.csv".
<code>sed 's/mot1/mot2/g' fichier.txt</code>	Remplace toutes les occurrences de "mot1" par "mot2" dans "fichier.txt".

Table: Exemples d'utilisations et descriptions (1/2)

<code>awk '{print \$1, \$3}' fichier.txt</code>	Affiche la première et la troisième colonne de "fichier.txt".
<code>sort fichier.txt</code>	Trie les lignes de "fichier.txt" par ordre alphabétique.
<code>sort fichier.txt uniq</code>	Supprime les doublons dans "fichier.txt" après tri.
<code>nl fichier.txt</code>	Affiche "fichier.txt" avec numérotation des lignes.
<code>split -l 10 fichier.txt</code>	Découpe "fichier.txt" en fichiers de 10 lignes chacun.
<code>date +" %Y-%m-%d %H:%M:%S"</code>	Affiche la date et l'heure au format spécifié.
<code>for i in {1..5}; .do echo "Nombre \$i"; done</code>	Affiche "Nombre 1" à "Nombre 5"
<code>seq 1 5</code>	Génère la liste des nombres de 1 à 5.

Table: Exemples d'utilisations et descriptions (2/2)

La programmation **Bash**

- combiner des commandes: (**cd**, **grep**, **tar**, **cut**, **awk**, **sed**)
- combiner des structures de contrôle (**if**, **for**, **while**, **case**)
- utiliser des variables etc
- automatiser des tâches répétitives, des scripts de démarrage etc.

Lancer un script

Un fichier exécutable contenant une série de commandes shell

- commence toujours sur la première ligne par **#!/bin/bash**
- doit être rendu exécutable (**chmod +x monscript.sh**)
- les autres lignes commençant par des **#** sont des commentaires
- s'exécute comme suit: **./monscript.sh**

Les variables en bash ne sont pas typées

- pas besoin de déclaration préalable comme en c ;
- pas de caractères spéciaux dans les noms des variables;
- les noms de variables ne peuvent commencer par un chiffre;
- Les variables d'environnement peuvent être utilisées (**\$HOME**).

L'affectation

Dans l'affectation il n'y a pas d'espace autour du signe "=". En dehors de l'affectation, pour son utilisation, la variable doit être précédée de \$

```
maVar=1 ; echo $maVar
```

```
maVar="Bonjour  ";
```

```
echo $maVar ou echo ${maVar}Jean
```


Les arguments ou paramètres

- Les arguments du script sont affectés aux variables \$1, \$2,...\$n.
Lancer un script avec paramètres **./monscript.sh Nom Prenoms**
#!/bin/bash
echo 'vous avez entre' \$1 \$2
- La variable \$? donne la valeur de sortie numérique de la dernière commande (**\$?=0 indique que la commande a réussi.**)
- \$# contient le nombre de paramètres passés en argument
- \$0 contient le nom du script exécuté

Les arguments

- Tester ce code avec plusieurs paramètres

```
./monscript.sh par1 par2 par3
```

```
#!/bin/bash
```

echo 'Vous avez lancé' \$0 et il y a \$# paramètres passé en argument, le premier paramètre est \$1 et voici toute la liste

```
for i in $@
```

```
do
```

```
echo $i
```

```
done
```

La substitution de commande ou la sous-exécution

La « sous-exécution » de commande permet de remplacer, lors de l'exécution du programme, une partie du script par le texte qu'une commande affiche normalement à l'écran. Pour ce faire on encadre la commande avec les **backquotes** ou avec **\$()**

- 1 `dateCourante=`date +%d%m%Y` ; echo $dateCourante.`

La variable `dateCourante` contient désormais le résultat de la commande `date +%d%m%Y`.

- 2 `DIR=$(pwd); cd $DIR/Documents;`

La variable `DIR` contient le résultat de la commande `pwd`

Les calculs arithmétiques

Le shell a priori n'est pas prévu pour faire de calculs mathématiques. Quatre méthodes de calcul s'offrent aux utilisateurs:

- 1 L'utilisation de `$(( ))`. Ex: `i=$((i+1))`.
- 2 L'utilisation de la commande `let`. Ex: `let "res=$a + $b"`
- 3 L'utilisation de la commande `expr` Ex: `res=$(expr $a "+" $b)`
- 4 L'utilisation de la commande `bc` Ex: `res=$(echo $a "+" $b|bc)`

De toutes les méthodes, seule **bc** permet des opérations à virgule flottante avec l'option `-l`. Ex: `f=$(echo "c(pi)/2"|bc -l)`.

Utiliser le manuel de `bc` pour plus de détails

Exemple de script shell: avec **expr** pour le calcul

```
#!/bin/bash
```

```
for V in $(seq 0 $2)
do
    echo $V " x " $1 " = " $(expr $V "*" $1)
done
```

Exemple de script shell: avec **let** pour le calcul

```
for V in $(seq 0 $2)
do
    let "res=$V "*" $1"
    echo $V "x" $1 "=" $res
done
```

Exercice: Utilisez les deux autres possibilités pour le même script

Exemple de script shell: sendVersion.sh

```
#!/bin/bash
```

```
DESTINATAIRE='jsagbo@gmail.com'
```

```
VERSION=3.0
```

```
REPERTOIRE=/home/ola/maquette-$VERSION
```

```
TAR=/tmp/maquette-${VERSION}.tgz
```

```
tar -czf $TAR $REPERTOIRE
```

```
mutt -a $TAR -s "maquette $VERSION" $DESTINATAIRE
```

```
«XX
```

```
Ci-joint la version $VERSION de la maquette
```

```
XX
```

Exemple de script shell - speedDownloadMarionnet.sh

```
#!/bin/bash
for i in $(seq 400)
do
    PROCESS=$(ps ax | grep marionnet | awk '{print $1}' | head -1)
    kill $PROCESS
    wget -c http://www.marionnet.org/download/testing/debian-wheezy/machine-debian-wheezy-54152&
    sleep 19
    echo $i
done
```

La structure de contrôle **if/then/else/fi**

- Permet d'effectuer des branchements conditionnels en fonction des variables

```
#!/bin/bash
echo 'Entrez un utilisateur'
read USER
if [ $USER = 'root' ]
then
    echo 'Bonjour maître'
else
    echo 'Encore un utilisateur simple ...'
fi
```

- La construction [] est l'équivalent de la commande **'test'** **Attention** il faut un espace après et avant chaque crochet [\$i -le \$j]

La commande **test** ou **[]** comparaison numérique

- Permet de comparer les valeurs entres elles
- **Comparaison numérique**
 - **-eq** : égal(**equal**)
 - **-ne** : different (**not equal**)
 - **-gt** : plus grand que (**greater than**)
 - **-ge** : plus grand que ou égal(**greater or equal**)
 - **-lt** : plus petit que (**less than**)
 - **-le** : plus petit que ou égal (**less or equal**)

La commande **test** ou **[]** comparaison de chaînes

- **=** : égal(**chaînes identiques**)
- **!=** : différent
- **<** : précède (tri alphabétique)
- **>** : suit (tri alphabétique)
- **-z** : longueur de la chaîne égal à 0
- **-n** : longueur de la chaîne différent de 0

la commande **test** ou **[]** comparaison des fichiers

- **-d** : Le fichier existe et est un répertoire
- **-e** : Le fichier existe
- **-f** : Le fichier existe et est un fichier
- **-r** : Le fichier existe et a un droit de lecture
- **-w** : Le fichier existe et a un droit d'écriture
- **-x** : Le fichier existe et est exécutable
- Pour plus de détails lire le manuel de **test**

Connecteurs logiques

- **-a** : «ET» logique
- **-o** : «OU» logique
- **!** : «NOT» logique
- **()** groupement d'expression doit être protégé de backslashes

Année Bissexile

```
y=$(date "+%Y") #récupération de l'année avec la commande date
if test \( $(expr $y % 4) -eq 0 -a $(expr $y % 100) -ne 0 \) -o $(expr $y % 400) -eq 0
then
    echo "l'année $y est une année bissextile"
fi
```

Exemple de script shell avec test sur un fichier - checkfiles.sh

```
#!/bin/bash  
FILE=/tmp/file.txt  
if test -e $FILE      # ou if [ -e $FILE ]  
then  
    echo "le fichier existe"  
else  
    echo "le fichier n'existe"  
fi
```

La structure de contrôle **case**

- Il est utilisé pour plusieurs options généralement dans les menus
- Sa syntaxe est comme suit:

```
#!/bin/bash
echo -n "Entrez un nombre : "
read nombre
case $nombre in
  1) echo "Vous avez tapé 1" ;; #attention aux doubles points virgules
  2) echo "Vous avez tapé 2" ;;
  *) echo "Vous n'avez tapé ni 1 ni 2" ;;
esac
```

Syntaxe de la boucle **select**

```
select var in item1 item2 ... itemn  
do  
commandes  
done
```

select est une boucle qui permet d'afficher de manière cyclique un menu. La liste des items, item1 item2 ... itemn, est affichée à l'écran à chaque tour de boucle. Les items sont indicés . La variable var est initialisée avec l'item correspondant au choix de l'utilisateur. Cette commande utilise également deux variables réservées:

- La variable PS3 représente le prompt utilisé pour la saisie du choix de l'utilisateur. Sa valeur par défaut est #?.
- La variable REPLY qui contient l'indice de l'item sélectionné.

```
#!/bin/bash
```

```
function creer_user { echo "Création de l'utilisateur" }  
function suppr_user {echo "Suppression de l'utilisateur" }  
function modif_user { echo "Modification des informations de l'utilisateur" }  
function fin {  
    echo "Sortie de Programme"  
    exit 0  
}  
PS3="Votre choix : "  
select item in "- Créer User -" "- Supprimer User -" "-Modifier les infos User- " "- Fin -"  
do  
echo "Vous avez choisi l'option $REPLY : $item"  
case $REPLY in  
    1) creer_user;;  
    2) suppr_user;;  
    3) modif_user;;  
    4) fin ;;  
    *) echo "Choix incorrect" ;;  
esac  
done
```


La boucle for

Syntaxe de la boucle **for**

```
for var in list  
do  
    commandes  
done
```

La boucle for peut encore utiliser la syntaxe du langage C comme suit:

```
#!/bin/bash  
for((i=0; i<=20; i++)) # attention aux doubles parenthèses  
do  
    date -d "+ $i years"  
done
```

Autres syntaxes de for (**for i in {a..z}** ou **for i in {0..10}**)

La boucle **until**

Tout comme la boucle **for**, la boucle **until** permet de boucler mais seulement jusqu'à ce qu'une condition soit remplie

```
#!/bin/bash
```

```
num=0
```

```
until [ num -gt 5 ]
```

```
do
```

```
    date -d +"$num years"
```

```
    ((num++))
```

```
done
```

on peut aussi utiliser **let num=\$num+1** en lieu et place de **((num++))**

La boucle **while**

La boucle while permet de boucler tant que la condition est vraie

```
ls | while read file
```

```
do
```

```
  if test -d "$file"
```

```
  then
```

```
    echo "$file est un répertoire"
```

```
  elif test -f "$file"
```

```
  then
```

```
    echo "$file est un fichier"
```

```
  else
```

```
    echo "$file est un fichier spécial ou lien symbolique ou pipe ou socket"
```

```
  fi
```

```
done
```

Les fonctions

La syntaxe des fonctions

```
nom_de_fonction()  
{  
    commande1  
    commande2  
}
```

Deuxième syntaxe avec le mot réservé **function**

```
function nom_de_fonction  
{  
    commande1  
    commande2  
}
```